FIT5196-S2-2026 Group Assessment Stage 1 (15%)

This is a group assessment and worth 15% of your total mark.

Due date: 23:55, Wednesday, Week 7 (9 September 2026)

This specification defines the assessed work, shared transformation rules, submission requirements, quick-start guidance, checklist and glossary. The separate marking rubric defines how evidence is evaluated. The public_data_dictionary.csv in your allocated data package defines the required output table grains, fields, field order, data types, nullability and comparison rules. The package README.md defines package-specific source conventions.

1. Scenario

Your team has received two historical exports from different operational systems used by a Melbourne technology retailer. One export is JSON and the other is XML. Together they describe customers, products, transactions, deliveries and long customer reviews.

1 2 3 4 5

The systems do not represent the same information in the same way. They use different nesting, field names, data formats and text conventions. Records may overlap within and across the two sources. Your team must build a reproducible workflow that parses the sources, reconciles overlapping records, produces six analysis-ready relational tables, validates the result and conducts a focused exploratory data analysis (EDA).

This is the first stage of the FIT5196 assessment project. Later assessment stages will use related current-period data. The teaching team will provide certified data for high-stakes downstream assessment so that an A1 mistake is not automatically charged again later. You should still retain your code, decisions and outputs because your reasoning may inform later work.

2. Learning outcomes assessed

By completing A1, your group should demonstrate that it can:

1. parse and inspect semi-structured JSON and XML using appropriate structured parsers;
2. map nested and repeated source records to tables with defined grains and relationships;
3. standardise identifiers, dates, timestamps, booleans, numeric values and narrative fields;
4. use regular expressions for bounded text cleaning and extraction tasks;
5. preserve valid multilingual Unicode text while creating a separate purpose-specific Latin-script analysis field;
6. reconcile duplicate and overlapping records without double counting;

7. validate keys, relationships, arithmetic and transformation behaviour;
8. conduct defensible univariate, bivariate and multivariate EDA; and
9. translate EDA evidence into feasible machine-learning questions while recognising leakage, evaluation and deployment risks.

Model training is not required in A1.

3. Files supplied

Each group receives one allocated archive named GroupNNN_A1.zip, where NNN is the three-digit group number. It contains:

- A1_manifest.json
- README.md
- public_data_dictionary.csv
- raw_input/
    - GroupNNN_commerce.json
    - GroupNNN_operations.xml

The teaching team also supplies the marking rubric and one templates/ folder:

- A1_source_to_target_mapping_template.csv;
- A1_solution_template.ipynb;
- A1_EDA_template.ipynb;
- A1_public_text_test_cases.csv; and
- A1_AI_use_declaration_template.docx.

The source files contain hundreds of customer and product records and thousands of transaction, delivery and review records. These are scale indicators, not expected clean row counts. The teaching team does not publish the canonical row counts, overlap counts, duplicate locations or expected output values.

Use only the data allocated to your group. Some source content may be shared across groups, but each package is assessed against its own certified reference. Before beginning, verify the group number in the archive name, source filenames and public manifest.

4. Required work

Task 1: Parse and profile the sources

Use a JSON parser for the JSON file and an XML parser for the XML file. Do not parse either structured document as plain text with regular expressions.

Your notebook and source-to-target mapping must show that you investigated the following. The report should summarise only the material decisions:

- major objects, nested arrays, repeated XML elements and source-specific representations;
- candidate primary keys and foreign keys;
- the grain of each source collection;
- date, timestamp, boolean, currency, percentage and missing-value formats;
- fields that appear in one or both sources;
- duplicate records within a source and overlapping records across sources; and
- assumptions needed before transformation.

Complete the supplied source-to-target mapping template. Its target fields are pre-filled, but you must investigate and enter the applicable JSON and/or XML path, derivation or normalisation, overlap/conflict rule and notebook evidence for every required output field. Source paths are intentionally not pre-filled because the two source layouts differ across allocated packages. The mapping must describe your method, not list every individual data row

Complete the six blank columns using JSON, XML, both or derived for source_format; structural paths for json_source_path and xml_source_path; and concise entries for transformation_or_derivation, overlap_or_conflict_rule and notebook_evidence. Use | to separate multiple input fields. The solution notebook template explains each column and contains fictional worked examples..

Task 2: Produce six standardised relational tables

Produce exactly these six submitted data tables:

1. orders
2. order_items
3. customers
4. deliveries
5. products
6. product_reviews

Use these filenames:

- GroupNNN_orders_standardised.csv
- GroupNNN_order_items_standardised.csv
- GroupNNN_customers_standardised.csv
- GroupNNN_deliveries_standardised.csv
- GroupNNN_products_standardised.csv
- GroupNNN_product_reviews_standardised.csv

The required grains and primary keys are:

Table	Grain	Primary key
orders	One row per canonical order	order_id
order_items	One row per order item	order_item_id
customers	One row per customer	customer_id
deliveries	One row per completed order delivery	delivery_id
products	One row per product	product_id
product_reviews	One row per canonical product review	review_id

Required foreign-key relationships are:

Child field	Parent field
orders.customer_id	customers.customer_id
order_items.order_id	orders.order_id
order_items.product_id	products.product_id
deliveries.order_id	orders.order_id
product_reviews.order_id	orders.order_id
product_reviews.order_item_id	order_items.order_item_id
product_reviews.product_id	products.product_id
product_reviews.customer_id	customers.customer_id

All listed primary and foreign keys are required and non-null.

For each file, follow public_data_dictionary.csv for table grain, field names, field order, data types, nullability and numeric comparison tolerance. Do not add helper or EDA-only columns to the six submitted CSV files. Helper tables and columns may be created within your code.

Your workflow must, where applicable:

- flatten nested arrays without changing the intended entity grain;
- retain valid one-to-many relationships rather than forcing all information into one flat table;

- normalise stable identifiers without inventing new entity identities;
- preserve identifier leading zeros and do not lower-case identifiers or structured categories unless a field-specific rule requires it;
- standardise dates as YYYY-MM-DD;
- standardise timestamps as YYYY-MM-DD HH:MM:SS;
- standardise booleans as True or False;
- remove currency labels and thousands separators before numeric conversion;
- represent percentages as the numeric percentage points required by the target field;
- derive fields required by the public dictionary, including line and order arithmetic where appropriate;
- use the literal string NaN for a prescribed missing string result rather than an empty string;
- reconcile records within and across sources using stable business keys and field-level comparison; and
- produce deterministic outputs when rerun on the same input.

For source reconciliation, normalise comparable fields first, compare records using the table primary key and retain one canonical row per key. Matching records in the assessed data are consistent after the published normalisation rules. A correctly normalised duplicate therefore does not require an arbitrary JSON-over-XML or XML-over-JSON precedence rule. If your code detects different non-missing values for the same target field and key, record the conflict in validation instead of silently choosing one.

For a prescribed missing string output, write the three literal characters NaN. This is not a Python None, NumPy/pandas missing value or empty field. When reading a submitted CSV for validation, consider pandas.read_csv(path, keep_default_na=False) so the sentinel remains visible. Do not place the string NaN in a required numeric or boolean field.

Use the following sequence for the published order arithmetic:

1. calculate each line_revenue as quantity * unit_price, rounded to two decimal places;
2. calculate order_price as the sum of the rounded line revenues, rounded to two decimal places;
3. treat product prices as GST-inclusive and calculate the included tax_amount as order_price / 11, rounded to two decimal places, before the coupon discount;
4. apply coupon_discount to order_price;
5. add delivery_charges; and
6. round the resulting order_total to two decimal places.

tax_amount is reported separately and is not added again to order_total. Use an absolute comparison tolerance of 0.01 for published monetary fields.

Do not hard-code canonical row counts, individual expected values or lists of known duplicate identifiers. A correct pipeline must derive its outputs from the allocated source files.

Task 3: Regex and multilingual text preprocessing

Regular expressions are assessed for bounded narrative cleaning and extraction. They must be applied after the relevant JSON value or XML element has been obtained with a structured parser.

Raw narrative fields may contain HTML entities, HTML/XML-like tags, URLs, emoji, mixed whitespace, social markers and bracketed system metadata. For this assessment, the complete set of removable bracketed and social marker patterns is::

[SYSTEM]

[CATALOGUE]

[VERIFIED_PURCHASE]

[SOURCE: ...]

[RATING: n/5]

#verified-buyer

@store_support

Apply this text-processing order:

1. obtain the value using the structured JSON/XML parser;
2. extract any required order, SKU or promotion reference from the raw value;
3. decode HTML entities and apply Unicode NFC normalisation;
4. remove HTML/XML-like tags while retaining human-readable content;
5. remove the listed system/source/rating/social markers, URLs and emoji;
6. remove a complete review reference wrapper, including Reference:, the order reference, separator, SKU: and SKU;
7. remove PROMO: together with its code;
8. collapse spaces, tabs and line breaks, trim, and lower-case the designated cleaned narrative; and
9. return the literal string NaN if no human-readable text remains.

Use regex for bounded patterns such as tags, markers, URLs, whitespace and business-reference extraction. Standard-library tools such as html and unicodedata may be used for entity decoding, Unicode normalisation and script classification. Do not use regex to reconstruct JSON or XML structure.

Extract these embedded business references when present:

Reference	Public format	Output when absent
Order reference	HORD or CORD followed by exactly six digits	NaN
Product SKU	SKU- followed by one or more ASCII letters or digits	NaN
Promotion code	B1SAVE- to B5SAVE-, followed by exactly two digits	NaN

Return extracted references in upper case. Boundary handling matters: a longer, malformed or embedded near-match must not be accepted as a valid reference.

Multilingual review contract

The reviews are primarily English but include a smaller multilingual component. The two review outputs have different purposes:

- review_body_clean must preserve valid multilingual UTF-8 letters. It must not erase a valid review merely because it contains non-Latin writing.
- review_body_latin_analysis is a separate derived field. It retains Latin-script letters, including European diacritics, together with applicable digits and punctuation, while removing non-Latin letters. If no Latin letter remains, return NaN.
- contains_non_latin_script is based on whether the cleaned multilingual review contains any letter outside the Latin script. A non-ASCII character is not automatically non-Latin.

Do not infer language_code from the Latin-analysis field. Use and reconcile the structured source evidence supplied for that attribute.

Build review_body_latin_analysis from review_body_clean, not from the noisy raw review. Derive the review measures as follows:

review_length_chars = number of Python characters in review_body_clean

review_word_count = number of whitespace-separated tokens in review_body_clean

When review_body_clean is the literal NaN, preserve the published sentinel behaviour rather than counting the three letters as an ordinary review.

Document and test your text functions with matched, unmatched, missing, multilingual and near-match examples. Run the supplied public text cases and add your own tests. Load the cases with keep_default_na=False when using pandas. The teaching team may test the submitted functions on private edge cases that follow this specification. Private tests will not require behaviour that is absent from the published rules.

Place the following six functions in GroupNNN_text_functions.py so the published interface can be tested without editing your notebook:

```
def clean_narrative_text(value):
    """Accept None or a string; return cleaned text or the string 'NaN'."""

def extract_order_reference(value):
    """Accept None or a string; return the upper-case reference or 'NaN'."""

def extract_product_sku(value):
    """Accept None or a string; return the upper-case SKU or 'NaN'."""

def extract_promo_code(value):
    """Accept None or a string; return the upper-case code or 'NaN'."""

def build_latin_analysis(value):
    """Accept cleaned multilingual text; return Latin analysis or 'NaN'."""

def contains_non_latin_script(value):
    """Accept cleaned multilingual text; return a Python bool."""
```

The function names, argument count and return contract are fixed. Your implementation is not. Do not perform file I/O, network access or row-specific lookups inside these functions.

Task 4: Validate the processed data

Provide executable validation checks in the validation-register section of the solution notebook. Give each check a stable ID such as VAL-PK-01so that the report can cite it without copying the complete check evidence. For every check, show the observed result, PASS/FAIL status and resolution or interpretation. The notebook template also reminds you of the difference between the literal string NaN, an empty string and a Python/pandas missing value. At a minimum, cover:

- required table and column presence;
- target data types and missing-value representation;
- primary-key completeness and uniqueness;
- foreign-key integrity between the six tables;
- allowed categorical values and sensible numeric ranges;
- source coverage and row counts before and after major transformations;
- handling of within-source and cross-source overlap;
- order price reconciliation against order-item lines;
- order total reconciliation against price, discount and delivery fields, plus a separate check that tax is the included GST component and was not added again;
- valid temporal ordering among order, dispatch, promised, delivered and review timestamps where applicable;
- extracted order/SKU/promotion references; and
- multilingual preservation and non-Latin-script indicators.

Use clear tolerances for floating-point or currency checks. A validation result must show the check, observed result and pass/fail status. Do not write assertions that only confirm a hard-coded expected count.

Finding and reporting a genuine source or contract issue is professional data work. A reported failure is not automatically incorrect when the evidence and proposed treatment are justified. Do not fabricate a value solely to make an assertion pass.

Task 5: Conduct focused EDA

Conduct univariate, bivariate and multivariate analysis across the relational outputs. Automated profile dumps do not by themselves satisfy this task.

Your report must include 6-8 clearly labelled assessed visualisations. The set must contain at least one investigation from each of these six categories:

1. univariate distribution or composition;
2. bivariate relationship or group comparison;
3. multivariate or segmented relationship;
4. temporal pattern;
5. review/text behaviour, such as language, length, rating or helpfulness; and
6. delivery or operational performance in relation to an order, customer, product or review characteristic.

Collectively, the assessed visualisations must use evidence from at least four of the six output tables and include at least two analyses that correctly join related tables. One visualisation may satisfy more than one category when the analytical question genuinely does so. A coordinated set of subplots counts as one visualisation when the panels answer one shared question. Label the assessed items Figure 1, Figure 2, and so on. Only the first eight clearly labelled assessed visualisations are marked.

For every assessed visualisation:

1. state the analytical question;
2. state the observation unit and denominator;
3. identify the tables and join keys used;
4. choose a chart appropriate to the variables and comparison;
5. provide readable titles, axes, legends and units; and
6. explain the result and a material limitation.

Take particular care with joins. Joining several one-to-many tables can multiply rows and inflate revenue, order or customer metrics. Calculate each metric at its intended grain and demonstrate a check against accidental double counting.

Task 6: Present findings and future ML questions

Present exactly ten numbered, evidence-based findings. Each finding must cite an assessed figure or reported statistic and explain:

- what was observed and at what grain;
- the magnitude, denominator or sample size;
- why it may matter in the retail context;
- a plausible alternative explanation or uncertainty; and
- a proportionate implication, next investigation or recommendation.

A finding that an apparent relationship is weak, uncertain or not decision relevant can be high quality when supported by evidence. Do not claim causation from observational association without a defensible design.

Propose exactly five numbered machine-learning questions grounded in your EDA. Together they must cover at least two different problem types, such as classification, regression, clustering, anomaly detection or forecasting. For each question, specify:

- the business decision or use case;
- the prediction or analysis unit;
- the proposed target or unsupervised objective;
- candidate predictors available at the decision time;
- an appropriate validation split;
- at least one evaluation metric; and
- likely target leakage, temporal leakage, fairness or deployment risks.

The questions must be feasible with the available type of data. Use the supplied EDA notebook template and aim for approximately 80-120 words or one compact table per question. Do not train or compare models in A1.

5. Reproducibility and code requirements

Submit Python code that recreates all six CSV files and validation results from the allocated raw package, plus an EDA notebook that recreates all assessed visualisations and reported statistics from those six CSVs.

Both submitted notebook must:

- contain a clearly labelled configuration cell; the solution notebook must define GROUP_ID, INPUT_DIR and OUTPUT_DIR;
- use relative or configurable paths rather than a student's absolute path;
- run from a fresh kernel using Restart and Run All;
- include the output required to review the analysis;
- avoid manual editing of generated CSV files;
- avoid hard-coded canonical answers; and
- use vectorised or otherwise reasonable methods for the supplied scale.

The solution notebook must contain the complete transformation and validation workflow. The EDA notebook must load the submitted six CSVs and recreate the reported figures and statistics without rerunning a different hidden cleaning pipeline. The submitted GroupNNN_solution.py export must reflect the solution notebook. It may import the required functions from GroupNNN_text_functions.py; the functions do not need to be duplicated. List any non-standard dependencies and versions in requirements.txt. The assessed workflow must run offline: it must not depend on a live AI service, external API, network download, private account, API key or unsubmitted local file.

Markers are not required to repair paths, install undocumented packages, rewrite student code or manually complete a failed transformation. Where a notebook cannot execute, available output and documented method may still provide partial evidence under the rubric.

6. EDA report requirements

Submit GroupNNN_EDA.pdf, with a maximum of 10 pages from the introduction to the conclusion. The cover page, contents page and references are excluded from the limit. Appendices are not assessed unless the specification explicitly requires the material or the main report directly cites it as validation evidence.

Use a clear structure such as:

1. problem context and data scope;
2. data-preparation assurance;
3. 6-8 assessed visualisations;
4. 10 numbered findings;
5. 5 numbered ML questions;
6. limitations; and
7. conclusion.

The data-preparation assurance section should communicate approximately 3-5 material transformation decisions and 4-6 material validation results, citing stable mapping and validation IDs where useful. The separate mapping CSV remains the complete field-lineage record, and the solution notebook remains the complete executable check record. Do not repeat either full artifact in the report.

Code listings do not belong in the report unless a short expression is necessary to explain an assessed decision. Tables and figures must remain readable at 100% zoom. Cite external facts, code, data and ideas using a consistent referencing style.

7. Submission requirements

Submit these two files to the A1 Moodle assignment:

- GroupNNN_A1_submission.zip

- GroupNNN_EDA.pdf

The ZIP file must contain:

- GroupNNN_solution.ipynb
- GroupNNN_solution.py
- GroupNNN_EDA.ipynb
- GroupNNN_text_functions.py
- GroupNNN_source_to_target_mapping.csv
- GroupNNN_AI_declaration.pdf
- requirements.txt # required only for non-standard dependencies
- AI_records/ # required when conversational AI was used
    - GroupNNN_AI_index.pdf
    - complete_chat_exports...
- outputs/
    - GroupNNN_orders_standardised.csv
    - GroupNNN_order_items_standardised.csv
    - GroupNNN_customers_standardised.csv
    - GroupNNN_deliveries_standardised.csv
    - GroupNNN_products_standardised.csv
    - GroupNNN_product_reviews_standardised.csv

Do not submit the raw JSON/XML files supplied by the teaching team. Do not place the EDA PDF inside the ZIP. Follow the checklist in Appendix A before uploading. Every group member is responsible for verifying that the correct group files appear in the Moodle submission receipt.

8. Generative AI use

Generative AI tools may be used while preparing A1, subject to University and unit policy. The group remains responsible for every submitted transformation, test, figure, citation, statement and file.

Every group must submit the signed AI-use declaration. If any member used a conversational AI tool, the group must also submit a complete direct export of every assignment-related conversation in AI_records/, together with an English index that records purpose, submission location and independent verification. Do not selectively shorten an assignment conversation. Use separate assessment-specific chats so unrelated private material is not mixed into the record. If a tool has no export function, print the complete conversation to PDF or HTML. Inline code completion with no conversational history must still be declared, but does not require an invented chat export. Follow templates/A1_AI_use_declaration_template.docx.

Appendix A: Submission checklist

Use this checklist after restarting and running the final notebook.

Group and package

- [ ] The allocated archive, manifest and raw filenames show the same GroupNNN identifier.
- [ ] Every submitted filename uses that exact three-digit identifier.
- [ ] Every group member has reviewed the final Moodle submission receipt.

Reproducibility

- [ ] GroupNNN_solution.ipynb and GroupNNN_EDA.ipynb both run using Restart and Run All.
- [ ] The notebook has configurable GROUP_ID, INPUT_DIR and OUTPUT_DIR values and no student-specific absolute paths.
- [ ] GroupNNN_solution.py reflects the final notebook workflow.
- [ ] GroupNNN_text_functions.py exposes all six required functions and performs no file I/O or network access.
- [ ] The solution workflow recreates the CSVs and checks; the EDA workflow recreates figures and reported statistics without manual edits or live services.
- [ ] requirements.txt is included if non-standard dependencies are required.

Data products and assurance

- [ ] All six required CSVs are inside outputs/.
- [ ] Filenames, columns, order, data types and grains match the public data dictionary.
- [ ] Primary keys, foreign keys, source coverage, arithmetic and time are checked.
- [ ] GST, discount, delivery and total follow the published calculation order.
- [ ] Duplicate reconciliation is data-driven rather than hard-coded.
- [ ] The mapping completes every pre-filled target row using stable MAP-... IDs.
- [ ] The solution notebook records stable VAL-... IDs, observed results, status, evidence and interpretation.

Text processing

- [ ] JSON and XML were parsed structurally before regex was applied to text.
- [ ] Tags, entities, markers, URLs, social tokens, emoji and whitespace follow the published rules.
- [ ] Order, SKU and promotion references have boundary and near-match tests.
- [ ] review_body_clean preserves valid multilingual text.
- [ ] The separate Latin-analysis field preserves Latin diacritics.
- [ ] Every supplied public text case was run and additional tests are shown.

Report

- [ ] GroupNNN_EDA.pdf is no more than 10 assessed pages.
- [ ] Between 6 and 8 figures are labelled as assessed visualisations.
- [ ] The set covers all six analytical categories, at least four tables and at least two correct relational analyses.
- [ ] Observation units, denominators and join-multiplication checks are clear.
- [ ] Exactly 10 evidence-based findings and 5 ML questions are numbered.
- [ ] Limitations, uncertainty, leakage risks and external citations are clear.

AI and final archive

- [ ] GroupNNN_AI_declaration.pdf is signed by all members.
- [ ] When conversational AI was used, AI_records/ contains the index and complete assignment-related exports.
- [ ] Inline completion is declared even when no chat export exists.
- [ ] Material AI-generated code and claims have independent checks.
- [ ] Raw teaching data is not included in the submission.
- [ ] The final uploaded files are exactly GroupNNN_A1_submission.zip and GroupNNN_EDA.pdf.

Appendix B: Glossary

Term	**Meaning in A1**
Canonical row	The single retained row representing one business entity after overlap is reconciled
Data grain	What one row represents, for example one order or one order item
Denominator	The total population used to calculate a rate or proportion
Deterministic	The same input and code produce the same output every time
Foreign key	A field that refers to a primary key in another table
Lineage	Where an output field came from and how it was transformed

Term	Meaning in A1
Near-match	Text resembling a valid reference but failing a required boundary
Observation unit	The entity measured in an analysis, such as an order or review
Primary key	Field or fields that uniquely identify one row at the table grain
Relational join	Combining tables through related key fields
Source-to-target mapping	A record of each output field's source and transformation
Target leakage	Using information unavailable when a prediction must be made
Temporal leakage	Using future information to predict an earlier event
Unicode	A standard representing writing systems and symbols across languages
Vectorised operation	Applying an operation to a column/array rather than looping over every row